

Cache Mastery & The Out-of-Order Revelation

We go deeper into how caches are organized to be robust, how to measure and optimize for them, and then tear open the black box of what happens when a processor stops obeying the order instructions are written in.

Set-Associativity: The Fix for Conflict Misses

A **set-associative cache** allows each memory address to map to one of N possible cache slots within a "set" (called an N-way set-associative cache). A 4-way set-associative cache has 4 possible "ways" (slots) per set — a memory address can land in any of the 4 slots in its set.

The three points on the associativity spectrum:

- **Direct-mapped (1-way):** one slot per set. Fastest lookup (one comparison), most conflict misses.
- **N-way set-associative:** N slots per set, N tag comparisons (done in parallel using N comparators). Typical: 4-way or 8-way in L1, 8-way to 16-way in L2/L3.
- **Fully associative:** any address can go in any slot. Zero conflict misses. But requires a comparator for every slot — impractical for large caches. Used in small TLBs and victim caches.

Higher associativity reduces conflict misses but increases cost (more hardware for parallel tag comparison) and slightly increases hit time. The sweet spot for most L1 caches is 4-way. Modern L3 caches are often 16-way.

Replacement Policies: Who Gets Evicted?

When all ways of a set are full and a new line must be loaded, which existing line is evicted? Three common policies:

- **LRU (Least Recently Used):** evict the line that was accessed least recently. Optimal in theory (exploits temporal locality), but tracking "least recently used" requires extra state bits. For 4-way caches, LRU adds 2 bits per set; for 16-way, it becomes expensive. Modern CPUs often use pseudo-LRU approximations.
- **Random:** evict a randomly chosen way. Surprisingly effective in practice (within 5% of LRU) and trivially cheap in hardware.
- **FIFO:** evict the oldest-loaded line. Simple, but ignores access patterns — a recently-fetched line that's never used again won't be evicted until it's the oldest.

Write Policies: When Data Goes Back to Memory

Caches hold copies of memory data. When the CPU writes to a cached address, two questions arise: when does the write go to memory, and what happens on a write miss?

Write-Hit Policies

- **Write-through:** every write immediately propagates to the next memory level. The cache and memory are always consistent. Simple. But generating a memory write on every cache write is slow and saturates the memory bus.
- **Write-back:** writes go only to the cache. A **dirty bit** marks the modified line. The line is written back to memory only when it's evicted. Most modern caches use write-back — it reduces memory traffic dramatically (a cache line may be written 50 times before eviction, but only 1 memory write occurs).

Write-Miss Policies

- **Write-allocate:** on a write miss, fetch the line into cache, then modify it (consistent with write-back).
- **No-write-allocate:** on a write miss, write directly to the next memory level without bringing the line into cache (consistent with write-through).

Almost all modern L1/L2 caches use write-back + write-allocate. The L3 is often write-back. Main memory is the final destination.

The AMAT Formula: Quantifying Cache Performance

Average Memory Access Time (AMAT)

$$\text{AMAT} = \text{HitTime_L1} + \text{MissRate_L1} \times (\text{HitTime_L2} + \text{MissRate_L2} \times \text{MissPenalty_L2})$$

This extends to any number of levels. Miss penalty at each level = the cost to fetch from the next level.

Worked example

L1: hit time = 1 cycle, miss rate = 5%

L2: hit time = 10 cycles, miss rate = 1% (of all accesses, not just L1 misses)

DRAM: latency = 100 cycles

$$\begin{aligned} \text{AMAT} &= 1 + 0.05 \times (10 + 0.01 \times 100) \\ &= 1 + 0.05 \times (10 + 1) \\ &= 1 + 0.05 \times 11 \\ &= 1 + 0.55 = \mathbf{1.55 \text{ cycles}} \end{aligned}$$

Without caches: $AMAT = 100$ cycles. The cache reduced effective memory latency by $65\times$.

The Devastating Impact of Cache-Unfriendly Code

The best demonstration of cache effects is matrix multiplication. Consider a matrix A ($N \times N$) stored in row-major order. Multiplying $A \times A$ requires accessing A by both row and column.

```
// Cache-UNFRIENDLY: column-major access of inner loop
for (i=0; i<N; i++)
    for (j=0; j<N; j++)
        for (k=0; k<N; k++)
            C[i][j] += A[i][k] * B[k][j]; // B[k][j]: j is inner → column walk!

// Cache-FRIENDLY: transpose B first, then row-major access
for (i=0; i<N; i++)
    for (k=0; k<N; k++)
        for (j=0; j<N; j++)
            C[i][j] += A[i][k] * B[k][j]; // inner loop: both row-major now
```

For a 1024×1024 matrix on a typical laptop: the unfriendly version takes ~ 8 seconds. The friendly version (or blocked/tiled version) takes ~ 0.8 seconds — a $10\times$ speedup with **zero algorithmic change**. The only difference is memory access order. This is what the memory hierarchy means in practice.

Teaser: Out-of-Order Execution

The setup

Everything so far has assumed a fundamental rule: instructions execute in the order the programmer wrote them. But what if the hardware could break that rule — safely — to go faster? That question is the heart of out-of-order execution, and it turns out the answer is: yes, it can, and it's one of the biggest sources of modern CPU performance.

Consider a program with 8 instructions. In a simple in-order pipelined processor, instructions enter the pipeline in order and stall whenever a dependency isn't yet satisfied. Let's trace exactly what this costs.

The Program

```
; Goal: compute (A+B) + (C+D) and store it
; Assume LOAD has 3-cycle latency (result available 3 cycles after issue)
; ADD has 1-cycle latency
```

```

I1: LOAD  R1, [A]      ; R1 = A
I2: LOAD  R2, [B]      ; R2 = B
I3: ADD   R3, R1, R2   ; R3 = A+B ← needs R1 AND R2
I4: LOAD  R4, [C]      ; R4 = C
I5: LOAD  R5, [D]      ; R5 = D
I6: ADD   R6, R4, R5   ; R6 = C+D ← needs R4 AND R5
I7: ADD   R7, R3, R6   ; R7 = final ← needs R3 AND R6
I8: STORE [E], R7      ; write result

```

In-Order Execution — Cycle by Cycle

Model: instruction issues at cycle T , result available at cycle $T + \text{latency}$. In-order means next instruction can issue at earliest $T+1$ (or when its operands are ready, whichever is later).

Instruction	Issue Cycle	Result Ready	Stall Reason
I1: LOAD R1,[A]	1	4	—
I2: LOAD R2,[B]	2	5	In-order, no dependency on I1
I3: ADD R3,R1,R2	5	6	Needs R2 ready at cycle 5. Stall 2 cycles
I4: LOAD R4,[C]	6	9	In-order after I3
I5: LOAD R5,[D]	7	10	In-order after I4
I6: ADD R6,R4,R5	10	11	Needs R5 ready at cycle 10. Stall 2 cycles
I7: ADD R7,R3,R6	12	13	Needs R6 ready at cycle 11. Stall 1 cycle
I8: STORE [E],R7	13	14	Needs R7 ready at cycle 13
Total	14 cycles to complete — 5 wasted stall cycles		

A critical observation: I4 and I5 (load C and D) have **absolutely no dependency** on I1, I2, or I3. While I3 stalls waiting for R1 and R2, the CPU could have been loading R4 and R5 — but an in-order processor is forbidden from doing so.

Out-of-Order Execution — The Hardware Can See Further

An OoO processor has a structure called the **instruction window** (or issue queue / reservation stations) — it looks ahead at many instructions simultaneously and issues any instruction whose operands are ready, regardless of program order. Results are still committed to the register file and memory in program order (via a structure called the **Reorder Buffer**), so the external behavior is identical. But internally, the hardware can overlap independent work.

Instruction	OoO Issue Cycle	Result Ready	Why this cycle?
I1: LOAD R1,[A]	1	4	No deps, issue immediately
I2: LOAD R2,[B]	1	4	Independent — issue same cycle as I1
I4: LOAD R4,[C]	1	4	Independent — OoO issues out of order!
I5: LOAD R5,[D]	1	4	Independent — all 4 loads fire at once
I3: ADD R3,R1,R2	4	5	R1 and R2 both ready at cycle 4
I6: ADD R6,R4,R5	4	5	R4 and R5 both ready at cycle 4
I7: ADD R7,R3,R6	5	6	R3 and R6 both ready at cycle 5
I8: STORE [E],R7	6	7	R7 ready at cycle 6
Total	7 cycles to complete — 2× speedup over in-order!		

The four LOAD instructions have no dependencies on each other. In-order execution forces them to run sequentially even though the hardware could execute them simultaneously. OoO execution sees the dataflow graph — not the instruction order — and executes based on what's ready, not what's next. This is also why memory latency is less catastrophic in OoO processors: while waiting for one load, the processor issues other independent work. The memory latency is effectively hidden.

The flip side: OoO requires complex hardware — reservation stations to hold waiting instructions, a reorder buffer (ROB) to track in-flight instructions and commit them in order, and register renaming to eliminate false WAR and WAW hazards. This is why each transistor in a modern superscalar OoO core does far more than just execute an instruction. Details about the actual implementation of out of order execution are left as an exercise to you! It isn't as simple as it looks.